Lab 3: Architecture Justification - Podcast Guest Scheduling & Outline Builder

Aryan Upadhyay | SRN: PES1UG25CS806 | Section C | Software Engineering (UE24CS341A)

Architecture Selection: "We chose a Layered Architecture for the Podcast Guest Scheduling & Outline Builder System."

Scenario review: hosts publish slots, guests book and submit talking points, and the host exports a timestamped run-of-show (FR-001...FR-005). Key challenges: **security** of guest data and booking links (NFR-002), **performance** of PDF generation under 1 s (NFR-001), and **usability** for two different user roles.

Architectural style analysis

Style	Fit for this system	Main drawback here
Layered (chosen)	One web application serving hosts and guests; clear split between UI, scheduling/outline rules and episode data; easy to test and maintain.	Small overhead of passing calls through layers; whole app is deployed together.
Microservices	Independent scaling is not needed for one show's booking and outline volume.	Many deployments, network hops and cross-service consistency for bookings - complexity with little benefit.
Client-Server	Matches the deployment (browsers talk to one server).	Says nothing about how the server is organised; on its own it tends to mix UI, rules and data access.

Components

Web Portal (Presentation); **Booking Manager**, **Outline Manager**, **Calendar Integration**, **Run-of-Show Generator** (Business); **Episode Repository** (Data). The handout's example components map to this scenario as: Order Manager -> **Booking Manager** (coordinates each booking) and Payment Service -> **Calendar Integration** (the one service that talks to an external provider); the given Order Manager <-> Payment Service link becomes Booking Manager <-> Calendar Integration (ICalendarSync).

Two reasons for this choice

1. **The system's concerns separate naturally into layers.** Two user roles share one web portal (presentation), the scheduling, outline and run-of-show rules form the business layer, and episodes, slots and outlines are stored data. With a moderate, predictable load (a show books a few guests a week), one layered application is simpler to build and deploy than a set of microservices.
2. **External dependencies are isolated in single components.** Only Calendar Integration talks to the calendar provider, only Run-of-Show Generator renders PDFs, and only the Episode Repository touches the database. The calendar provider can be switched (Google to Outlook) without changing booking rules, and each layer can be tested with the layer below mocked - for example, the timestamp calculation is unit-tested without a database.

Security advantage

Every request from the web portal must pass through the business layer before reaching data. Booking Manager checks that a booking-link token is valid and unexpired, and Outline Manager checks that the logged-in host owns the episode, before the Episode Repository is called. The UI can never query the database directly, and the calendar OAuth tokens live only inside Calendar Integration. This enforces NFR-002 (unguessable, expiring links; guest details visible only to the episode's host) in one place.

Performance benefit

The business components run in the same server process, so Outline Manager hands the already-loaded episode data to Run-of-Show Generator as an in-process call: one database query, an in-memory timestamp calculation, then the PDF render. This keeps PDF generation under the 1-second target (NFR-001). Calendar invites are sent asynchronously by Calendar Integration, so a slow external calendar API never delays the guest's booking confirmation.